

开发任务一操作指南

在通信之前，我们需要事先在通信双方约定一个通信语法，这样子才不会出现鸡同鸭讲的情况。现在我们需要制定一个通信语法也就是对应着一个报文格式标准，用来指定传输过程中数据的组织形式。报文格式标准没有固定的要求，唯一的要求就是发送端和接收端都接受相同的标准。数据报文（Packet）总体来说分为两个部分：报文头部和包内数据。报文头部主要包含描述报文特性的一些元数据，例如报文序列号、报文的发送时间等等，类似我们日常生活中寄快递时的快递单。报文头部的格式需要我们仔细设计，因为报文大小是受限的，头部的空间占用的越多，发送有效数据可用的空间就会越少。我们给出了一个简单的数据报文格式标准，见源文件中 `contest_message.hh`。

```
C++
string payload;
struct Header header;
```

注 1：本协议适用的场景是固定发送端和接收端，因此不需要在报文头元数据部分添加源 ip、源端口、目的 ip、目的端口。若有多个发送、接收端或者发送端和接收端不固定的情况下，需要在报文头部分添加源地址和目的地址信息。

下面我们将会展开报文头部的字段设计，报文头包含报文的元数据信息。因为我们的协议需要支持可靠传输，所以我们必须在报文头部分增加一个序列号元数据(sequence_number)，用来跟踪哪些报文已经发出。其次，我们还必须在报文头部增加发送时间元数据(send_timestamp)，用来记录报文发出的时间。现在我们的数据报文格式变为了：

```
C++
struct Header {
    uint64_t sequence_number;
    uint64_t send_timestamp;
} header;
```

```
string payload;
```

以上是发送端所需的一些元数据，但是要实现可靠传输还需要接收端对接收到的数据进行反馈，所以还需要设计一种接收端使用的确认报文（ACK）。简要起见，我们在代码中复用了数据报文的报文格式，但是还需要针对接收端适当的添加一定的元数据来满足确认报文的要求。

首先，确认报文也有自身的序列号元数据（sequence_number），用来标识已经确认收到了多少个数据报文。同时也需要记录确认的数据报文序列号元数据

（ack_sequence_number），用来表示该确认报文响应的数据报文。另外，确认报文也需要记录发送时间元数据（send_timestamp），但同时也需要包含数据报文被发送的时间元数据（ack_send_timestamp）和数据报文被接收的时间元数据

（ack_recv_timestamp）。需要注意这两个时间基于的时钟是不同的，其中数据报文被发送的时间是基于发送端的时钟，数据报文被接收的时间是基于接收端的时钟。有了这两个时间理论上发送端就可以根据返回的 ACK 携带的信息准确的计算出 RTT

（Round Trip Time），进一步地发送端可以根据 RTT 进行拥塞控制等操作。同时，因为可靠传输需要保证接收端收到的数据是有序的，所以理想情况下接收端反馈 ACK 的时候，需要反馈接收到的数据报文中包含的数据范围，包括数据报文中有效载荷首所在发送端数据流的位置（start）以及数据报文有效载荷的长度（offset）。但是在本项目代码中，发送端发送的数据是无意义的，所以严格跟踪数据报文的数据范围并不十分的有效。简要起见，我们只需要在 ACK 中反馈数据报文有效载荷大小元数据

（ack_payload_length），从而让发送端感知到已被确认的数据长度。现在我们的数据报文格式变为了：

```
C++
struct Header {
    uint64_t sequence_number;
    uint64_t send_timestamp;

    uint64_t ack_sequence_number;
    uint64_t ack_send_timestamp;
    uint64_t ack_recv_timestamp;
    uint64_t ack_payload_length;
} header;
string payload;
```

注 2: ACK 中的 `payload` 和数据报文中的 `payload` 虽然属性名称相同, 但实际有很大区别。数据报文中的 `payload` 代表的是发送端传输的有效数据, 是由应用层给出的, 一般长度较大。而 ACK 中一般不需要 `payload`, 长度为 0。当然也有学者认为 ACK 中可以携带一些额外的信息来实现一些特殊的机制, 具体可参考 TACK: Improving Wireless Transport Performance by Taming Acknowledgments 这篇论文。

我们虽然在 ACK 的格式上复用了 `Packet` 的格式, 但是在以上的报文格式标准中, 并没有对他们做明确的区分, 所以在实际发送或接收的过程中会产生歧义, 因此需要对报文格式做出适当的修改。

开发任务:

请仿照以上修改报文格式的过程在报文格式中添加一个新的元数据, 比如报文类型元数据, 并使用新添加的报文类型元数据修改 `contest_message.hh` 中的数据结构, `contest_message.cc` 中的相关的方法, 并最后修改 `receiver.cc` 实现打印新添加的元数据功能。

提交要求:

提交修改后的 `contest_message.hh`, `contest_message.cc`, `receiver.cc` 以及设计说明一份。代码要求一人一码, 严禁抄袭!

设计说明书要求:

要求写清楚任务目标、任务实施过程中的细节, 包括但不限于算法细节描述、机制原理说明和过程结果截图等。